

Compressed Sensing and Sparse Recovery - HW6

Tran Duong Chinh - v202603119

2025-05-18

Table of contents

Part 1

1

Part 1

Implement the face-recognition algorithm as described in Lecture based on the concept of Sparse Representation Classification (SRC).

The database that we will use to evaluate the performance of your algorithms is the Extended Yale Face Database B which contains 38 human subjects under different poses and illumination conditions. To reduce the computational complexity, we will only experiment with the cropped database whose images have been manually aligned, cropped, and then re-sized to 96×84 . This database can be downloaded from Canvas as a mat file.

```
# Import the necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat
from tqdm import tqdm

from HW2 import solve_l1_linear_programming, solve_orthogonal_matching_pursuit,
    plot_solution
```

```
# Load the data from .mat files
DATA_FILE_PATH = '../data/CroppedYale_96_84_2414_subset.mat'

data = loadmat(DATA_FILE_PATH)

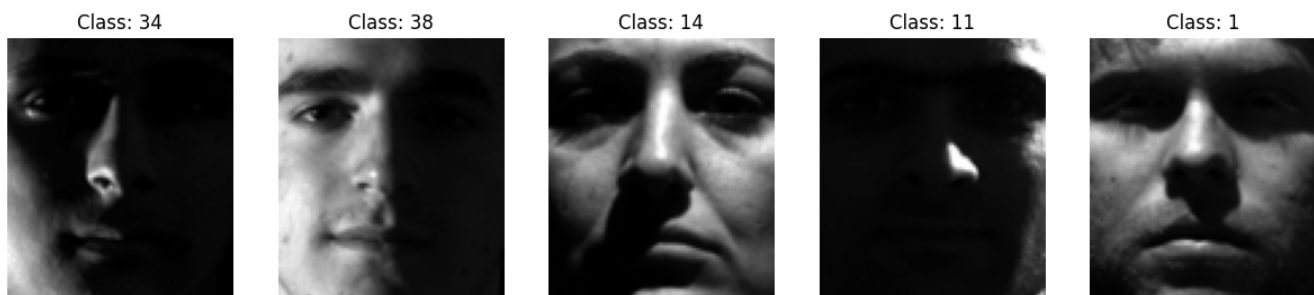
print("Data loaded successfully!")
print(data.keys()) # Print the keys to understand the structure of the loaded data
```

```
Data loaded successfully!
dict_keys(['__header__', '__version__', '__globals__', 'sizeim', 'faces', 'facecls',
'facesubset'])
```

```
print(data['faces'].shape)
print(data['facecls'].shape)
print(data['facesubset'].shape) # 5 different subsets
```

```
(2414, 96, 84)
(2414, 1)
(2414, 1)
```

```
# Visualize some sample images from the dataset
num_samples = 5
plt.figure(figsize=(15, 3))
for i in range(num_samples):
    id = np.random.randint(0, data['faces'].shape[0]) # Randomly select an index
    plt.subplot(1, num_samples, i + 1)
    plt.imshow(data['faces'][id, :, :], cmap='gray')
    plt.title(f"Class: {data['facecls'][id][0]}")
    plt.axis('off')
plt.show()
```



```
# Split the data into training and testing sets
train_subsets = [0, 1, 2] # Use subsets 0, 1, and 2 for training
test_subsets = [3, 4] # Use subsets 3 and 4 for testing

train_indices = np.isin(data['facesubset'], train_subsets).flatten()
test_indices = np.isin(data['facesubset'], test_subsets).flatten()

X_train = data['faces'][train_indices, :, :]
y_train = data['facecls'][train_indices].flatten()
X_test = data['faces'][test_indices, :, :]
y_test = data['facecls'][test_indices].flatten()
```

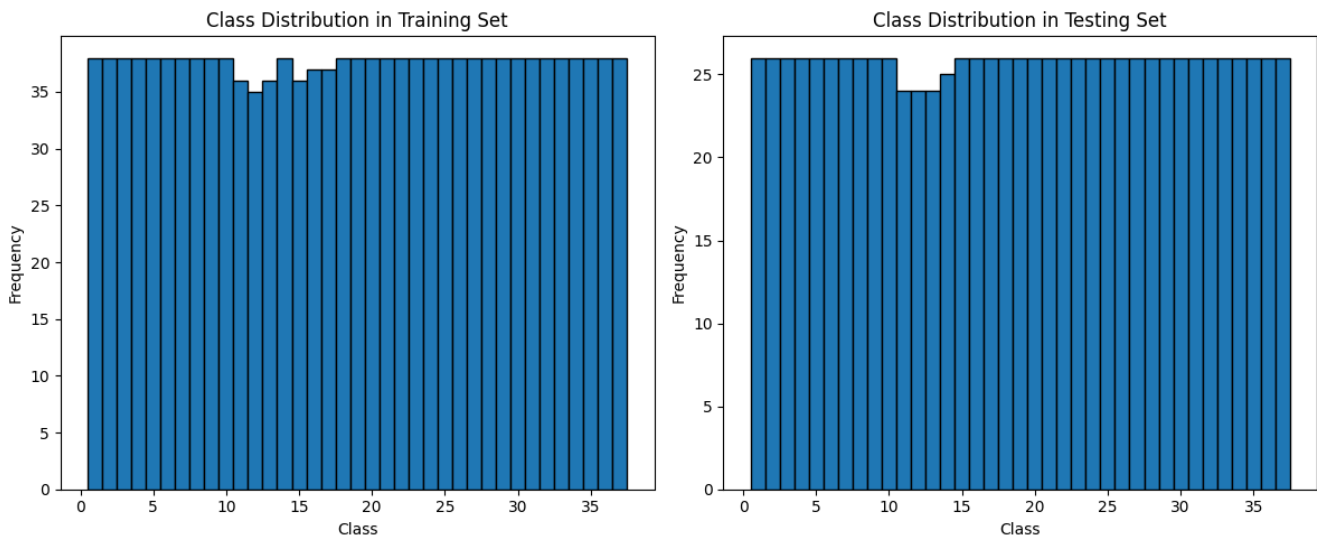
```
y_train[:30], y_test[:30] # Print the first 10 labels of training and testing sets
```

```
(array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1], dtype=uint8),
 array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 2, 2, 2, 2], dtype=uint8))
```

```
# Plot the distribution of classes in the training and testing sets
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.hist(y_train, bins=np.arange(1, 39) - 0.5, edgecolor='black')
plt.title('Class Distribution in Training Set')
plt.xlabel('Class')
plt.ylabel('Frequency')
```

```
plt.subplot(1, 2, 2)
plt.hist(y_test, bins=np.arange(1, 39) - 0.5, edgecolor='black')
plt.title('Class Distribution in Testing Set')
plt.xlabel('Class')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```
# Build a map of class labels and their corresponding indices in the training set
class_to_indices = {}
for cls in np.unique(y_train):
    class_to_indices[int(cls)] = np.where(y_train == cls)[0]
print(class_to_indices.keys())
```

```
dict_keys([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21,
22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38])
```

```
# Build a dictionary for sparse representation
```

```
# For simplicity, we will use the training images as the dictionary
# Each column of the dictionary will be a vectorized training image
```

```
num_train_samples = X_train.shape[0]
num_features = X_train.shape[1] * X_train.shape[2] # Assuming grayscale images
dictionary = X_train.reshape(num_train_samples, -1).T # Shape: (num_features,
↪ num_train_samples)
```

```
# Perform classification one test image using sparse recovery methods (L1 minimization
↪ and OMP)
```

```
test_image = X_test[0, :, :].flatten() # Vectorize the first test image
test_image_label = y_test[0]
print(f"Test image label: {test_image_label}")
```

```
# L1 minimization
```

```

# sparse_code_l1 = solve_l1_linear_programming(dictionary, test_image)

# OMP
sparse_code_omp = solve_orthogonal_matching_pursuit(dictionary, test_image,
    ↪ max_iter=100) # 100 iterations for OMP to save time

# Print the non-zero coefficients and their corresponding class labels for both methods
# non_zero_indices_l1 = np.where(sparse_code_l1 != 0)[0]
non_zero_indices_omp = np.where(sparse_code_omp != 0)[0]

# print("L1 Minimization - Non-zero coefficients and their corresponding class
    ↪ labels:")
# for idx in non_zero_indices_l1:
#     print(f"Coefficient index: {idx}, Class label: {y_train[idx]}")
print("\nOMP - Non-zero coefficients and their corresponding class labels:")
for idx in non_zero_indices_omp[:5]: # Display only the first 5 non-zero coefficients
    print(f"Coefficient index: {idx}, Class label: {y_train[idx]}")

```

Test image label: 1

OMP - Non-zero coefficients and their corresponding class labels:

```

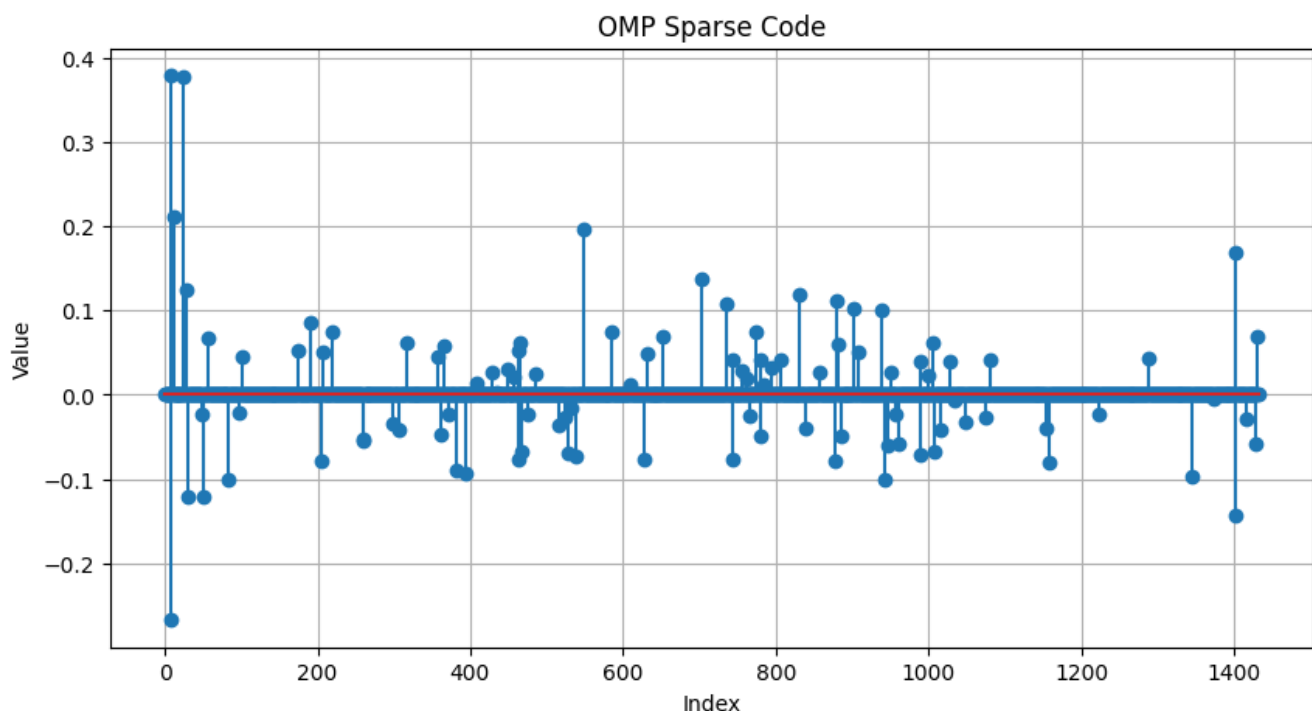
Coefficient index: 7, Class label: 1
Coefficient index: 8, Class label: 1
Coefficient index: 12, Class label: 1
Coefficient index: 25, Class label: 1
Coefficient index: 29, Class label: 1

```

```

# Plot the sparse codes
plot_solution(sparse_code_omp, title="OMP Sparse Code")

```



```

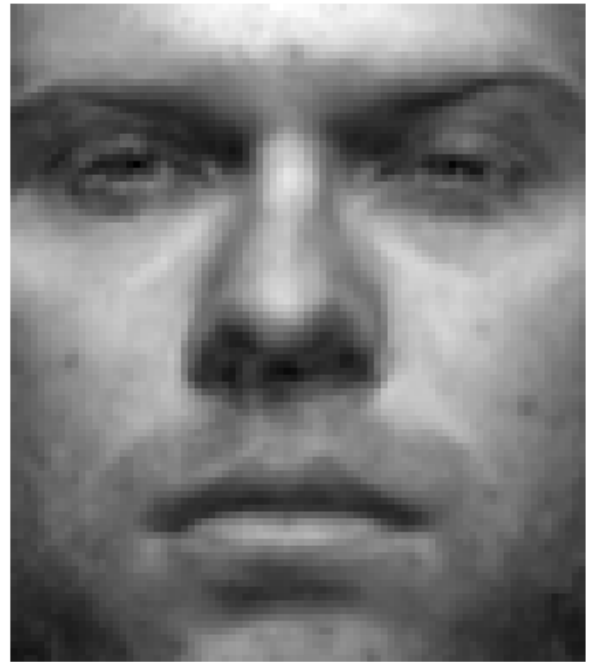
# Plot the test image and the reconstructed image from OMP
reconstructed_image_omp = dictionary @ sparse_code_omp
reconstructed_image_omp = reconstructed_image_omp.reshape(X_train.shape[1],
    ↪ X_train.shape[2]) # Reshape to original image dimensions
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(test_image.reshape(X_train.shape[1], X_train.shape[2]), cmap='gray')
plt.title(f"Original Test Image (Class: {test_image_label})")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(reconstructed_image_omp, cmap='gray')
plt.title("Reconstructed Image from OMP")
plt.axis('off')
plt.show()

```

Original Test Image (Class: 1)



Reconstructed Image from OMP



We can get pretty good reconstruction just using 100 iterations of OMP.

```

# To find the predicted class, we can look at the non-zero coefficients and their
    ↪ corresponding class labels.
# For each class, we mask the coefficients that does not belong to that class and
    ↪ compute the residual error. The class with the smallest residual error can be
    ↪ considered as the predicted class.

def compute_residual_error(test_image, dictionary, class_id):
    """
    Reconstruct the image using only the coefficients that belong to the specified
    ↪ class and compute the residual error.

    Parameters:

```

```

test_image (numpy array): The original test image (vectorized).
dictionary (numpy array): The dictionary of training images (shape: num_features x
↪ num_train_samples).
class_id (int): The class ID for which to compute the residual error.
Returns:
residual_error (float): The computed residual error for the specified class.
reconstructed_image (numpy array): The reconstructed image using the coefficients
↪ of the specified class.
"""
# Create a mask for the current class
class_mask = (y_train == class_id)

# Mask the coefficients that do not belong to the current class
masked_sparse_code = np.copy(sparse_code_omp)
masked_sparse_code[~class_mask] = 0

# Reconstruct the image using the masked sparse code
reconstructed_image = dictionary @ masked_sparse_code

# Compute the residual error
residual_error = np.linalg.norm(test_image - reconstructed_image)

reconstructed_image = reconstructed_image.reshape(X_train.shape[1],
↪ X_train.shape[2]) # Reshape to original image dimensions

return residual_error, reconstructed_image

def predict_class(sparse_code, dictionary, y_train):
    unique_classes = np.unique(y_train)
    residuals = {}

    for cls in unique_classes:
        # Create a mask for the current class
        class_mask = (y_train == cls)

        # Mask the coefficients that do not belong to the current class
        masked_sparse_code = np.copy(sparse_code)
        masked_sparse_code[~class_mask] = 0

        # Reconstruct the image using the masked sparse code
        reconstructed_image = dictionary @ masked_sparse_code

        # Compute the residual error
        residual_error = np.linalg.norm(test_image - reconstructed_image)
        residuals[cls] = residual_error

    # Find the class with the smallest residual error
    predicted_class = min(residuals, key=residuals.get)
    return predicted_class, residuals

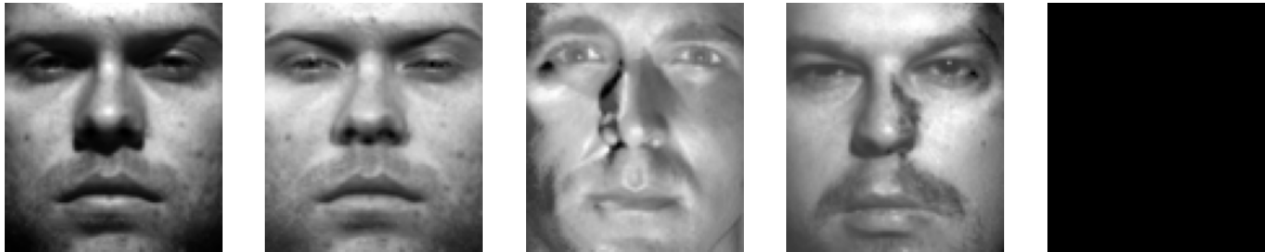
```

```

# Compare the reconstructed image with the original test image and compute the residual
↪ error for a few classes
classes_to_compare = [1, 10, 20, 30] # Example class IDs to compare
figure, axes = plt.subplots(1, len(classes_to_compare) + 1, figsize=(15, 5))
axes[0].imshow(test_image.reshape(X_train.shape[1], X_train.shape[2]), cmap='gray')
axes[0].set_title(f"Original Test Image (Class: {test_image_label})")
axes[0].axis('off')
for i, cls in enumerate(classes_to_compare):
    residual_error, reconstructed_image = compute_residual_error(test_image,
↪ dictionary, cls)
    axes[i + 1].imshow(reconstructed_image, cmap='gray')
    axes[i + 1].set_title(f"Class {cls} - Residual: {residual_error:.2f}")
    axes[i + 1].axis('off')
plt.show()

```

Original Test Image (Class: 1) Class 1 - Residual: 3594.52 Class 10 - Residual: 10554.29 Class 20 - Residual: 9948.15 Class 30 - Residual: 11021.32



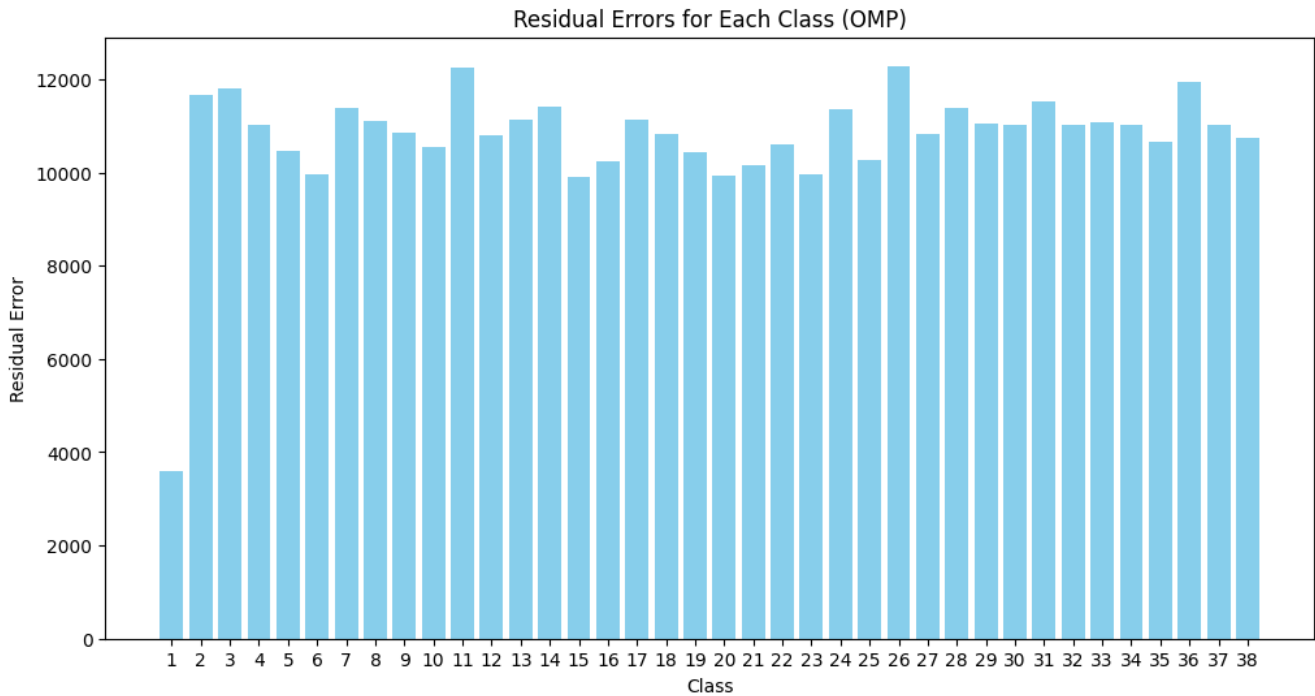
```

# Predict the class of the test image using OMP coefficients
predicted_class_omp, residuals_omp = predict_class(sparse_code_omp, dictionary,
↪ y_train)
print(f"Predicted class using OMP: {predicted_class_omp}")

# Plot the residual errors for all classes
plt.figure(figsize=(12, 6))
plt.bar(residuals_omp.keys(), residuals_omp.values(), color='skyblue')
plt.xlabel('Class')
plt.ylabel('Residual Error')
plt.title('Residual Errors for Each Class (OMP)')
plt.xticks(list(residuals_omp.keys()))
plt.show()

```

Predicted class using OMP: 1



```
# Test all test images and compute the overall accuracy
correct_predictions_omp = 0
for i in tqdm(range(X_test.shape[0])):
    test_image = X_test[i, :, :].flatten()
    test_image_label = y_test[i]

    # Compute the sparse code for the test image using OMP
    sparse_code_omp = solve_orthogonal_matching_pursuit(dictionary, test_image,
    ↪ max_iter=100)

    # Predict the class using the computed sparse code
    predicted_class_omp, _ = predict_class(sparse_code_omp, dictionary, y_train)

    if predicted_class_omp == test_image_label:
        correct_predictions_omp += 1

# Compute the overall accuracy
accuracy_omp = correct_predictions_omp / X_test.shape[0]
print(f"Overall accuracy using OMP: {accuracy_omp:.4f}")
```

```
0%|          | 0/981 [00:00<?, ?it/s]100%|          | 981/981 [1:03:02<00:00, 3.86s/it]
```

Overall accuracy using OMP: 0.9959

To run the entire test set took over 1 hour on my machine. To move on to the next part I think